

# microPAM\_V4 — Classifier Development Summary

**Date:** 2026-06-28

**Branch:** main

**Version:** 4.2.0

---

## 1. Overview

This document summarises the development of the online VAE-based signal classifier for microPAM\_V4, carried out entirely in the Python simulator (Python/simulator.py) with firmware changes applied once a configuration was validated.

The goal is to detect transient acoustic events (e.g. bat echolocation chirps) in real-time on a Teensy 4.1, classify which frequency sub-band they occupy, and log the result without interrupting recording.

---

## 2. Processing Chain (PROC\_MODE 4)

```
acq_buffer (int32, 2048 samples, 4-ch interleaved)
├ spectrum_apply()    50%-overlap RFFT per channel → Z[512, 4, 2]
├ intensity_apply()   Im(conj(Za)·Zb) cross-correlations → I[512,3], D[512]
├ detection_apply()   exponential background tracking → Dsnr
├ classifier_apply()  4 parallel VAEs → vae_mu[16], vae_detect[4]
└ queue.push()        16 float32 latent means → SD card (.vae file)
```

### Key constants

| Symbol        | Value      | Meaning                             |
|---------------|------------|-------------------------------------|
| FSAMP         | 192 000 Hz | Sample rate                         |
| NBUF_ACQ      | 2048       | Samples per DMA buffer (4 ch × 512) |
| NSAMP         | 512        | Samples per channel per buffer      |
| NFFT          | 1024       | FFT length (50% overlap)            |
| DETECT_THR    | 3.0        | SNR threshold for signal detection  |
| DETECT_ALPHA  | 0.01       | Background EMA coefficient          |
| VAE_NVAE      | 4          | Number of parallel VAEs             |
| VAE_QSAMP     | 128        | D[] bins per VAE (freq sub-band)    |
| VAE_H1        | 32         | Hidden layer width                  |
| VAE_LAT       | 4          | Latent dimensions per VAE           |
| VAE_LAT_TOTAL | 16         | Total exported latent means         |
| LR            | 1e-4       | SGD learning rate                   |
| BETA          | 1e-3       | KL regularisation coefficient       |

### VAE frequency band mapping

| VAE  | D[] bins | Frequency range |
|------|----------|-----------------|
| VAE0 | 0-127    | 0-24 kHz        |
| VAE1 | 128-255  | 24-48 kHz       |
| VAE2 | 256-383  | 48-72 kHz       |
| VAE3 | 384-511  | 72-96 kHz       |

### 3. VAE Architecture

Each of the 4 VAEs is an independent online VAE with architecture:

$Q=128 \rightarrow \text{ReLU}(W1 @ x + b1) \rightarrow (\mu, \text{lv}) \rightarrow z = \mu + \text{eps} \cdot \exp(0.5 \cdot \text{lv})$   
 $\rightarrow \text{ReLU}(W3 @ z + b3) \rightarrow W4 @ h3 + b4 \rightarrow \text{recon}[Q]$

- Loss:  $\text{MSE}(\text{recon}, D\_q) + \beta \cdot \text{KL}$ ,  $\text{KL} = 0.5 \cdot \sum (\exp(\text{lv}) + \mu^2 - 1 - \text{lv})$
- Weights initialised Xavier uniform; biases zero.
- $W1[4][32][128]$  and  $W4[4][128][32]$  placed in DMAMEM (128 KB total).
- All other weights in RAM1 (~17 KB).

### 4. Noise-Gated Training

**Key design decision:** The VAE is trained **only on noise frames** ( $\text{Dsnr} < \text{DETECT\_THR}$ ). On signal frames the backward pass is skipped; only the forward pass runs to extract  $\mu$ .

This prevents the signal events from corrupting the noise background model that the VAE latent space encodes.

**Implementation (C — classifier.cxx):**

```
bool signal_frame = (Dsnr_buf >= DETECT_THR);
for (int v = 0; v < 4; v++) {
    vae_forward(v, D_q);
    if (!signal_frame) vae_backward(v, D_q);
}
if (signal_frame) {
    for (int k = 0; k < VAE_LAT_TOTAL; k++) vae_mu_signal[k] += vae_mu[k];
    vae_mu_signal_count++;
}
```

Dsnr is passed from `dsp_apply()` to `classifier_trigger()` as an extra argument and stored in `Dsnr_buf` for the async ISR to read.

**Exported globals (readable from .ino):**

| Variable                         | Type                 | Description                         |
|----------------------------------|----------------------|-------------------------------------|
| <code>vae_mu[16]</code>          | <code>float[]</code> | Current-frame latent means          |
| <code>vae_mu_signal[16]</code>   | <code>float[]</code> | Accumulated means on signal frames  |
| <code>vae_mu_signal_count</code> | <code>uint32</code>  | Number of accumulated signal frames |

The .ino monitor prints and resets `vae_mu_signal / count` every second.

---

## 5. Per-VAE Detection Score

**Simulator-only** (Python/simulator.py). No firmware equivalent.

Each VAE maintains a noise background EMA for both `mu` and reconstruction error, updated only on noise frames (same gate as training):

```
# noise frame only:
mu_bg[v] += DETECT_ALPHA * (mu_v - mu_bg[v])
recon_bg[v] += DETECT_ALPHA * (err_v - recon_bg[v])

# every frame:
mu_dev = ||mu_v - mu_bg[v]||2
recon_ratio = recon_v / recon_bg[v]
vae_detect[v] = mu_dev × recon_ratio
```

`vae_detect[v]` is large when the current frame's latent representation has moved away from the noise cluster **and** the reconstruction error has increased — i.e. both components must agree that something unusual is present.

---

## 6. Hyperparameter Sweep Results

Sweep over 50 000 iterations per configuration (`noise_amp = 225`, `USE_SYNTH_SIGNAL=True`):

| Option            | VAE_H1 | VAE_LAT  | BETA | LR   | max sep  $\sigma$ | mean sep  $\sigma$ |
|-------------------|--------|----------|------|------|-------------------|--------------------|
| Baseline          | 32     | 2        | 1e-3 | 1e-4 | 2.64              | 0.53               |
| Lower BETA=1e-4   | 32     | 2        | 1e-4 | 1e-4 | 2.62              | 0.52               |
| Lower BETA=1e-5   | 32     | 2        | 1e-5 | 1e-4 | 2.62              | 0.52               |
| <b>LAT=4</b>      | 32     | <b>4</b> | 1e-3 | 1e-4 | <b>17.48</b>      | <b>2.36</b>        |
| Wider H=64        | 64     | 2        | 1e-3 | 1e-4 | 5.04              | 1.00               |
| Higher LR=1e-3    | 32     | 2        | 1e-3 | 1e-3 | 1.16              | 0.28               |
| LAT=4 + BETA=1e-5 | 32     | 4        | 1e-5 | 1e-4 | 17.48             | 2.35               |
| LAT=4 + H=64      | 64     | 4        | 1e-5 | 1e-4 | 24.13             | 2.45               |

**Selected: VAE\_LAT=4, VAE\_H1=32** — LAT=4 gives the largest gain; H=64 is excluded because  $W1+W4$  at H=64 requires 256 KB DMAMEM, overflowing the Teensy 4.1 RAM2 (only 12 KB headroom). `VAE_LAT_TOTAL` becomes 16 (was 8). BETA and LR unchanged.

---

## 7. VAE Input Selection

Three options tested over 100 000 iterations:

| Input                   | VAE0 recall  | VAE1 recall  | VAE2 recall  | VAE3 recall  | Notes        |
|-------------------------|--------------|--------------|--------------|--------------|--------------|
| <b>Intensity D[]</b>    | <b>83.7%</b> | <b>70.8%</b> | <b>47.4%</b> | <b>44.8%</b> | Best overall |
| Mean spec-trum (all ch) | 86.5%        | 49.9%        | 48.8%        | 51.6%        | Mixed        |
| Spectrum ch0 only       | 56.4%        | 44.5%        | 40.7%        | 43.0%        | Worst        |

**Selected: intensity D[]** — cross-channel intensity suppresses uncorrelated noise (random inter-channel phases cancel), giving the VAE a higher-SNR input than any single-channel or averaged power spectrum.

## 8. Confusion Matrix (final configuration, 100 000 iterations)

VAE input: intensity D[], VAE\_LAT=4, noise-gated training, noise\_amp=2<sup>25</sup>.

| GT \ Pred | noise | VAE0 | VAE1 | VAE2 | VAE3 |                |
|-----------|-------|------|------|------|------|----------------|
| noise     | 89762 | 1865 | 1817 | 1826 | 733  | (recall 93.5%) |
| VAE0      | 153   | 837  | 5    | 5    | 0    | (recall 83.7%) |
| VAE1      | 13    | 279  | 707  | 0    | 0    | (recall 70.8%) |
| VAE2      | 1     | 0    | 524  | 474  | 0    | (recall 47.4%) |
| VAE3      | 21    | 0    | 1    | 529  | 448  | (recall 44.8%) |

**Combined detector** (sum of vae\_detect across 4 VAEs, threshold = noise\_mean + 3σ): - True positive rate: **93.6%** (3747 / 4003 signal frames) - False positive rate: **0.80%** (770 / 95 997 noise frames)

## 9. Timing Structure of VAE Detection

The synthetic chirp sweeps 80 kHz in one 2.67 ms buffer, so all 4 VAE frequency bands receive energy simultaneously. However, because each SYNTH step starts at a different f0, the peak vae\_detect for each VAE occurs at a different frame:

SYNTH step 0 (f0= 3 kHz) → VAE0 peaks at event frame + 0..1  
 SYNTH step 1 (f0= 27 kHz) → VAE1 peaks at event frame + 1..2  
 SYNTH step 2 (f0= 51 kHz) → VAE2 peaks at event frame + 2..3  
 SYNTH step 3 (f0= 75 kHz) → VAE3 peaks at event frame + 3..4

The 1-frame sequential offset between VAEs is **accepted as part of the design** — it reflects the physical sweep of the chirp through frequency and is not a defect. The resulting confusion between adjacent VAEs (VAE2↔VAE3) is a known limitation.

## 10. Why VAE0 Scores Lower Than VAE1-3

The directional intensity estimator computes:

$$D[k] = ||\text{Im}(\text{conj}(Z_a) \cdot Z_b)|| \quad \text{for channel pairs } (a,b)$$

At low frequencies (VAE0: 0–24 kHz), inter-channel delays [0, 20, 10, 10] samples produce small phase differences ( $\Delta\phi = 2\pi \cdot f \cdot \Delta t \ll 1$ ), making  $\text{Im}(\text{conj}(Z_a) \cdot Z_b) \approx 0$  even for a coherent signal. VAE0 is therefore in the directional sensitivity null. This is physically correct for the given array geometry — it is not a bug.

Mean D excess per band on signal frames: - VAE0 (0–24 kHz): 0.0010 (4× lower) - VAE1 (24–48 kHz): 0.0041 - VAE2 (48–72 kHz): 0.0040 - VAE3 (72–96 kHz): 0.0030

---

## 11. Files Modified / Created

| File                        | Change                                                                                                       |
|-----------------------------|--------------------------------------------------------------------------------------------------------------|
| src/classifier.h            | VAE_LAT 2→4; added vae_mu_signal[], vae_mu_signal_count; updated classifier_trigger signature                |
| src/classifier.cxx          | Noise-gated training; Dsnr_buf; vae_mu_signal accumulation                                                   |
| src/process.cxx             | Passes Dsnr to classifier_trigger                                                                            |
| microPAM_V4.ino             | Prints vae_mu_signal/count in monitor loop; resets after print                                               |
| Python/simulator.py         | Full processing chain mirror; noise-gated VAE; vae_detect; per-frame log; confusion matrix; OceanNoise class |
| Python/viewer.py            | Time-series viewer (Dsnr + vae_detect); confusion matrix heatmap                                             |
| Python/synth_spectrogram.py | Spectrogram of synthetic chirp signals per step and channel                                                  |
| Python/decompress.py        | Decoder for compressed .bin/.spc/.int files                                                                  |
| Python/micropam_reader.py   | File reader dispatching by PROC_MODE                                                                         |
| Python/sim_log.npz          | 100 000-frame simulation log (not versioned)                                                                 |

---

## 13. Realistic Ocean Noise Model (Wenz 1962)

### 13.1 Background

Flat white Gaussian noise is an unrealistic acoustic background for ocean deployments. The OceanNoise class added to Python/simulator.py replaces the white noise generator with spectrally shaped noise whose PSD follows the empirical Wenz (1962) model, parameterised by **ship density** (shipping traffic level) and **sea state** (Beaufort scale).

Reference: <https://www.passiveacoustics.org/2025/03/16/ambient-noise/>

### 13.2 Wenz Model Components

The total noise PSD (dB re 1  $\mu\text{Pa}^2/\text{Hz}$ ) is the linear (power) sum of four components:

| Component    | Equation (f in kHz)                                                                                                                      | Dominates       |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Turbulence   | $NL_{\text{turb}} = 17 - 30 \cdot \log_{10}(f)$                                                                                          | < 10 Hz         |
| Shipping     | $NL_{\text{ship}} = 20 + 12w + 20 \cdot \log_{10}(f) - 30 \cdot \log_{10}(f^2 + (15 + 4w)/10^4)$                                         | 10 Hz - 1 kHz   |
| Wind/Knudsen | $NL_{\text{surf}} = 44 + 28 \cdot \log_{10}(1 + v^{0.75}) + 19 \cdot \log_{10}(f) - 18 \cdot \log_{10}(f^2 + 1/(2.78 + 0.14v^{0.75})^2)$ | 100 Hz - 50 kHz |
| Thermal      | $NL_{\text{therm}} = -15 + 20 \cdot \log_{10}(f)$                                                                                        | > 50 kHz        |

Sea state S (Beaufort 0-9) converts to wind speed:  $v = 27.6 \cdot (S/10)^{1.4}$  m/s. Ship density  $w \in \{0, 1, 2\}$  = light / moderate / heavy traffic.

### 13.3 Implementation (OceanNoise class)

```
noise_src = OceanNoise(ship_density=1, sea_state=3,
                       rms_counts=2**25, rng=rng)
acq_buffer = noise_src.generate() # int32, shape (NBUF_ACQ,), channels interleaved
```

The class pre-computes the amplitude spectrum at `__init__` time (one rfft over NSAMP bins), then per `generate()` call produces independent per-channel coloured noise: white noise  $\rightarrow$  FFT  $\rightarrow$  multiply by amplitude filter  $\rightarrow$  IFFT  $\rightarrow$  scale to rms\_counts RMS.

Module-level switches in `simulator.py`:

| Constant         | Default | Meaning                      |
|------------------|---------|------------------------------|
| USE_OCEAN_NOISE  | True    | Enable Wenz noise model      |
| SHIP_DENSITY     | 1       | 0=light, 1=moderate, 2=heavy |
| SEA_STATE        | 3       | Beaufort 0-9                 |
| NOISE_RMS_COUNTS | 2**25   | Overall RMS in int32 counts  |

### 13.4 Performance Comparison (100 000 frames each)

All runs: NOISE\_RMS\_COUNTS =  $2^{25}$ , USE\_SYNTH\_SIGNAL = True, VAE\_LAT = 4.

#### White noise (baseline)

| GT \ Pred | noise | VAE0 | VAE1 | VAE2 | VAE3 |              |
|-----------|-------|------|------|------|------|--------------|
| noise     | 89762 | 1865 | 1817 | 1826 | 733  | recall=93.5% |
| VAE0      | 153   | 837  | 5    | 5    | 0    | recall=83.7% |
| VAE1      | 13    | 279  | 707  | 0    | 0    | recall=70.8% |
| VAE2      | 1     | 0    | 524  | 474  | 0    | recall=47.4% |
| VAE3      | 21    | 0    | 1    | 529  | 448  | recall=44.8% |

Detector: TPR=95.3% FPR=6.50%

### Ocean noise — moderate (ship\_density=1, sea\_state=3, wind=5.1 m/s)

| GT \ Pred | noise | VAE0 | VAE1 | VAE2 | VAE3 |              |
|-----------|-------|------|------|------|------|--------------|
| noise     | 90365 | 1426 | 1767 | 1692 | 753  | recall=94.1% |
| VAE0      | 522   | 459  | 9    | 10   | 0    | recall=45.9% |
| VAE1      | 3     | 178  | 818  | 0    | 0    | recall=81.9% |
| VAE2      | 0     | 2    | 590  | 407  | 0    | recall=40.7% |
| VAE3      | 0     | 1    | 0    | 502  | 496  | recall=49.6% |

Detector: TPR=86.9% FPR=5.87%

### Ocean noise — heavy/rough (ship\_density=2, sea\_state=6, wind=13.5 m/s)

| GT \ Pred | noise | VAE0 | VAE1 | VAE2 | VAE3 |              |
|-----------|-------|------|------|------|------|--------------|
| noise     | 90280 | 1506 | 1771 | 1692 | 754  | recall=94.0% |
| VAE0      | 608   | 369  | 10   | 13   | 0    | recall=36.9% |
| VAE1      | 4     | 173  | 822  | 0    | 0    | recall=82.3% |
| VAE2      | 0     | 3    | 545  | 451  | 0    | recall=45.1% |
| VAE3      | 0     | 1    | 0    | 539  | 459  | recall=45.9% |

Detector: TPR=84.7% FPR=5.96%

### Summary

| Condition           | TPR          | FPR   |
|---------------------|--------------|-------|
| White noise         | <b>95.3%</b> | 6.50% |
| Ocean, ship=1, SS=3 | 86.9%        | 5.87% |
| Ocean, ship=2, SS=6 | 84.7%        | 5.96% |

### 13.5 Analysis

**VAE0 (0-24 kHz) degrades strongly under ocean noise.** The shipping component of the Wenz model dominates 10 Hz – ~1 kHz and rolls off as  $\sim f^{-2}$  up to ~24 kHz, exactly overlapping the VAE0 band. The VAE background model adapts to elevated, spectrally structured low-frequency noise, reducing its ability to separate the chirp from natural fluctuations.

| VAE  | White noise signal/noise ratio | Ocean SS3 ratio | Ocean SS6 ratio |
|------|--------------------------------|-----------------|-----------------|
| VAE0 | 8.2×                           | 4.5×            | 3.3×            |
| VAE1 | 14.8×                          | <b>83.5×</b>    | <b>84.2×</b>    |
| VAE2 | 14.4×                          | <b>110.3×</b>   | <b>121.6×</b>   |
| VAE3 | 7.2×                           | 15.2×           | 15.3×           |

**VAE1 and VAE2 (24-72 kHz) improve dramatically under ocean noise.** The Wenz spectrum rolls off sharply above ~5 kHz (shipping gone, thermal not yet dominant), leaving mid-high frequencies quieter than white noise. The VAE background EMA settles at a

lower level, producing a larger relative signal excursion and higher signal-to-noise ratios ( $83\text{--}121\times$ ).

**FPR is slightly lower under ocean noise** ( $6.5\% \rightarrow \sim 5.9\%$ ) for the same reason: a stable, predictable background in the mid-high bands produces tighter noise distributions and more accurate per-VAE thresholds.

**Practical implication:** for real ocean deployments with significant shipping traffic, VAE0 sensitivity will be reduced and may require a higher-SNR source signal, a longer detection integration window, or a dedicated low-frequency background model.

---

## 12. Known Limitations and Open Questions

1. **VAE2/VAE3 recall  $\sim 45\text{--}47\%$**  — caused by the 1-frame timing ambiguity between adjacent SYNTH steps. Accepted by design.
2. **False alarm rate  $0.80\%$  per frame** — at  $192\text{ kHz} / 512\text{ samples per frame} = 375\text{ frames/s}$ , this gives  $\sim 3$  false alarms per second. May need a 2-of-N gate for practical deployment.
3. **VAE0 low sensitivity** — directional null at low frequencies for the given array geometry. Could be addressed by a different array configuration or by using the spectrum instead of intensity for VAE0 only.
4. **Noise amplitude sensitivity** — detection requires  $\text{noise\_amp} \leq 2^{26}$  for the synthetic chirp to exceed DETECT\_THR. Real-world SNR must be verified on hardware.
5. **Online SGD instability at  $\text{LR}=1\text{e-}3$**  — confirmed in sweep; keep  $\text{LR}=1\text{e-}4$ .